# A Routing Algorithm Performance Simulator Using C++

Ahmed Bahgat
Computer Engineering Department

November 25, 2025

**Abstract**

Efficient routing is a fundamental requirement for modern computer networks. Different routing algorithms vary considerably in terms of optimality, control overhead, convergence behavior, and robustness. This project presents a **Routing Algorithm Performance Simulator** implemented in C++, whose purpose is to experimentally compare four major routing strategies: **Flooding**, **Shortest Path First (Dijkstra)**, **Distance Vector (Bellman–Ford)**, and **Link State**.

The simulator constructs a flat network topology, applies each routing algorithm separately, routes packets between selected source–destination nodes, and records several performance metrics such as hop count, path cost, number of control messages, and convergence steps. The resulting comparisons highlight the strengths and weaknesses of each approach under identical network conditions.

Arel Üniversitesi, Müh. Mim Fak. Bilgisayar Mühendisliği. Türkoba Mahallesi, Erguvan Sokak, No:26, K 34537, Tepekent – Büyükçekmece, İstanbul

# 1 Introduction

Routing is the process of selecting paths in a network so that packets can travel from a source to a destination. Different algorithms use different mechanisms to construct forwarding tables, maintain network knowledge, and react to topology changes. These mechanisms directly influence:

- Path optimality
- Delivery time
- Control overhead
- Convergence speed
- Scalability and robustness

This project aims to transform the theoretical routing algorithms studied in Data Communication courses into a practical and measurable simulation environment. Instead of studying each algorithm separately, the simulator runs all algorithms on the **same network** and compares their performance side by side.

The proposed simulator performs the following tasks:

- Builds a graph-based network topology of routers and weighted links
- Implements multiple routing algorithms
- Routes packets through the network
- Records performance metrics for each protocol
- Produces a final comparison table summarizing all results

This practical approach provides a clearer understanding of routing behavior and facilitates deeper analysis than purely theoretical study.

# 2 System Overview

The system consists of three main components:

- **Topology Module**: Generates routers, links, and network structure.

- **Routing Algorithms Module**: Implements Flooding, Shortest Path, Distance Vector, and Link State protocols.
- **Simulation Engine**: Runs routing, delivers packets, and collects performance metrics.

Unlike hierarchical or multi-layer routing, this simulator uses a **single flat network** to ensure all algorithms operate under equal conditions.

# 3 Design Objectives

The main design objectives of the Routing Algorithm Performance Simulator are:

- **Modularity**: Each routing algorithm is implemented as an independent component via a common interface.
- **Clarity**: Routing tables, packet paths, and metrics are displayed in a readable format.
- **Comparability**: All algorithms run on the same topology and test cases.
- **Educational Value**: The simulator demonstrates routing behaviors without requiring complex hardware.

# 4 System Design and Class Structure

The simulator follows an object-oriented design. Figure 1 shows the UML class diagram representing the main components of the system.

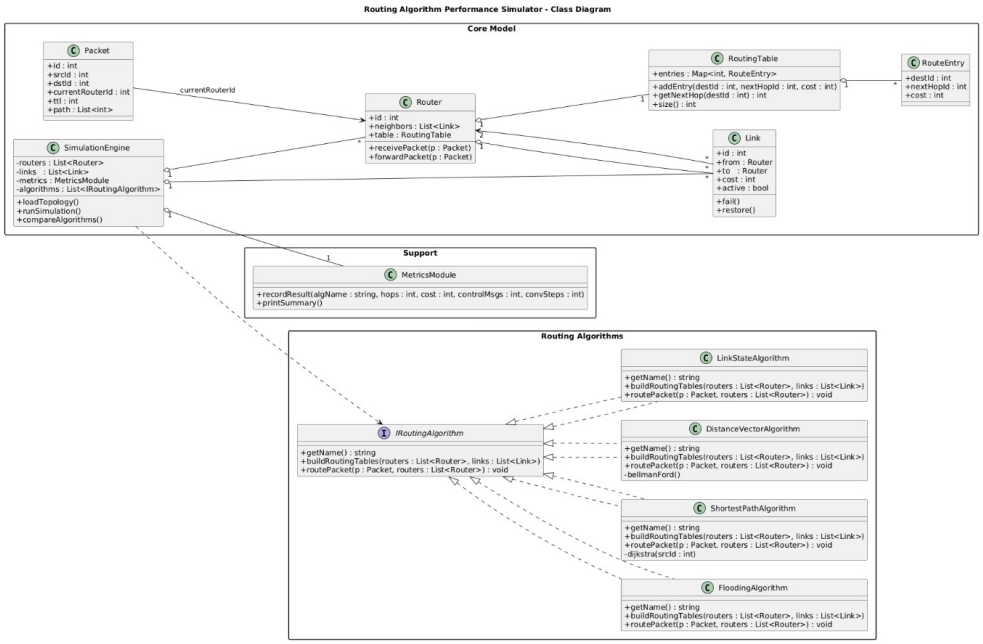



Figure 1: UML class diagram of the routing simulator components.

The `SimulationEngine` manages the overall execution by loading the topology, running each routing algorithm, and recording metrics. The `Router` and `Link` classes together represent the network graph. The `Packet` class models the path taken through the network. The `IRoutingAlgorithm` interface ensures all routing algorithms implement common functions for table building and packet forwarding.

# 5 Program Flow and Implementation Plan

Figure 2 shows the flowchart of the simulation logic. The engine loads the network, selects a routing algorithm, builds routing tables, executes packet forwarding, and logs results. This process is repeated for all algorithms to produce comparative metrics.

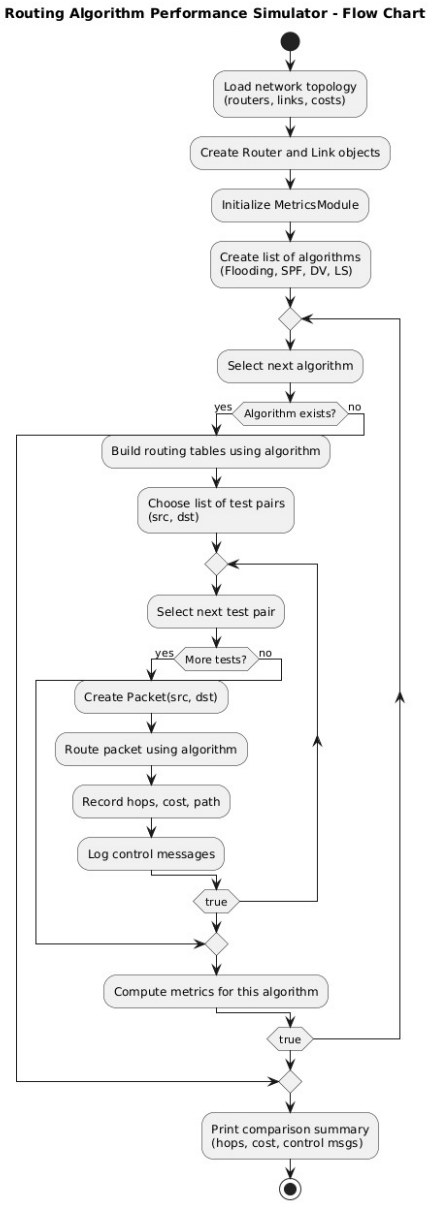



Figure 2: Flowchart of the Routing Algorithm Performance Simulator.

The implementation will follow these steps:

- Implement the basic `Router`, `Link`, `Packet` and `RoutingTable` structures.
- Implement the `FloodingAlgorithm`, `ShortestPathAlgorithm`, `DistanceVectorAlgorithm`, and `LinkStateAlgorithm`.
- Integrate all algorithms using the `IRoutingAlgorithm` interface.

- Develop the `MetricsModule` to track hops, costs, control messages, and convergence.
- Test with different topologies and evaluate performance.

# 6 Conclusion

This project introduces a modular Routing Algorithm Performance Simulator that allows students to observe the operational differences between several key routing algorithms. By implementing Flooding, Shortest Path, Distance Vector, and Link State routing in the same environment, the simulator reveals how each method behaves in terms of optimality, overhead, and convergence. The system provides a valuable educational tool for exploring routing concepts in a controlled and measurable setup.

# References

[1] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Prentice Hall, 2011.

[2] B. A. Forouzan, *Data Communications and Networking*, 5th ed. McGraw-Hill, 2012.

[3] R. Bellman, "On a routing problem," Quarterly of Applied Mathematics, 1958.

[4] E. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1959.

[5] J. Moy, "OSPF Version 2," RFC 2328, Internet Engineering Task Force, 1998.